INTRODUCTION TO PROGRAMMING • DIGITAL NOTES

UNIT I

Introduction to Programming & Problem Solving

Computers • Languages • Algorithms • Flowcharts • Data Types • I/O • Operators • Complexity

▶ What you will learn in this unit

- History & generations of computers
- Basic organization: ALU, CU, memory, PC, I/O
- Programming languages
- Basics of a computer program
- Algorithms, flowcharts (Dia) & pseudocode
- Compilation & execution
- Data types, variables & constants
- Basic input / output
- Operators, type conversion & casting
- Problem solving: top-down / bottom-up
- Characteristics of an algorithm
- Time & space complexity

1.1 History of Computers

◆ Definition

A **computer** is an electronic machine that accepts **data** (input), processes it using stored **instructions** (program), and produces **information** (output).

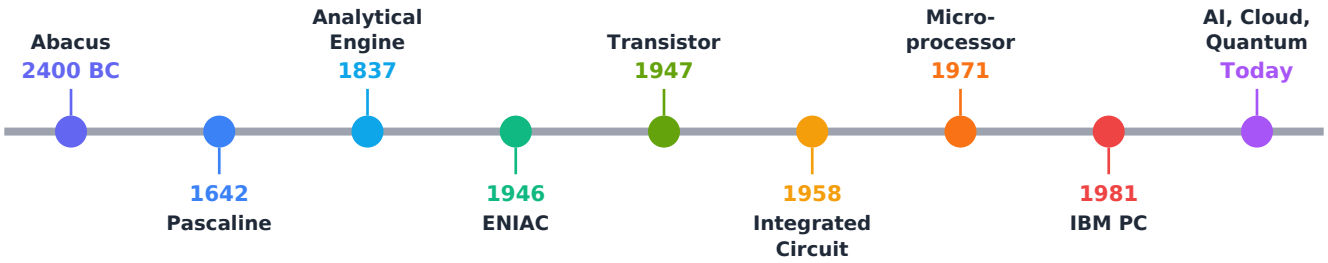


Fig 1.1 — Milestones in the history of computing

Gen.	Period	Technology	Features / Examples
1st	1940–56	Vacuum tubes	Huge, hot, slow, machine language — ENIAC, UNIVAC
2nd	1956–63	Transistors	Smaller, faster, assembly language — IBM 1401
3rd	1964–71	Integrated Circuits (IC)	High-level languages, operating systems — IBM 360
4th	1971–today	Microprocessor (VLSI)	Personal computers, GUI, networks — IBM PC, laptops
5th	Present & beyond	ULSI, AI, Quantum	Artificial intelligence, voice recognition, robotics

✔ Remember

Easy way to remember: Tubes → Transistors → ICs → Microprocessors → AI. Each generation made computers **smaller, faster, cheaper and more reliable**. *Charles Babbage* is the "Father of the Computer"; *Ada Lovelace* wrote the first program.

Characteristic	Meaning	Application area	Use
Speed	Billions of operations per second	Education	e-learning, online exams
Accuracy	Errors only from wrong input or program	Banking	ATMs, online payment
Storage	Stores huge data permanently	Healthcare	Scans, patient records
Automation	Runs without human help	Science	Simulation, satellites

1.2 Basic Organization of a Computer

Every computer, from a phone to a supercomputer, follows the **Von Neumann architecture**: programs and data are stored together in memory, and the CPU fetches and executes them one by one.

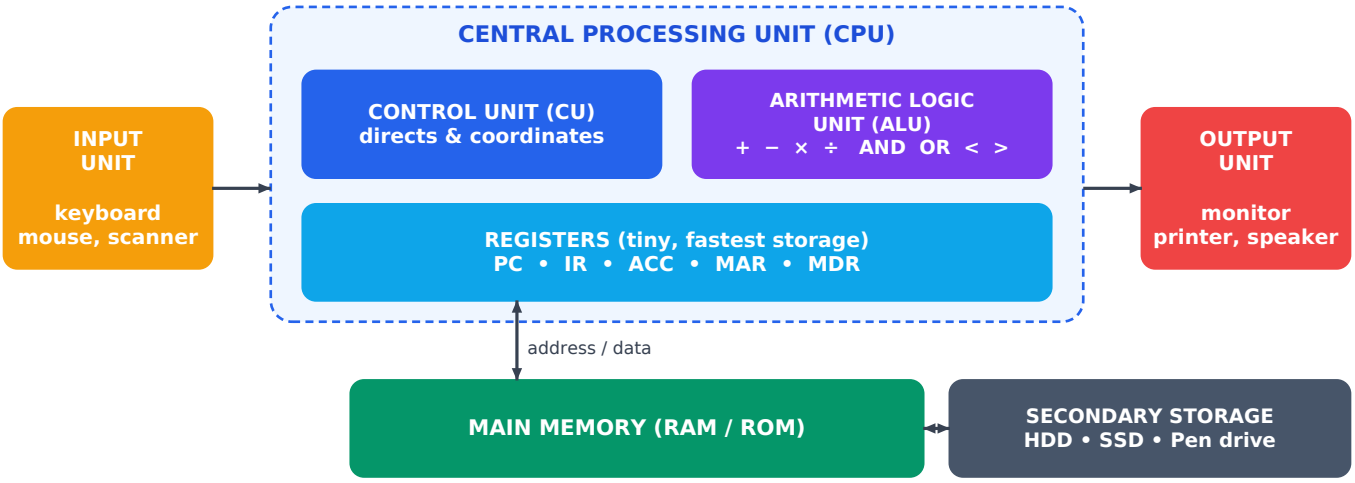


Fig 1.2 — Block diagram of a computer system

Unit	Function	Example
Input unit	Takes data/instructions from the user and converts them into binary	Keyboard, mouse, scanner, mic
ALU	Performs **arithmetic** (+ − × ÷) and **logical** (AND, OR, NOT, comparisons) operations	Adds two numbers, checks a > b
Control Unit	Fetches instructions, decodes them, and sends control signals to all other parts	The "manager" of the CPU
Memory	Holds programs and data. **Primary** (RAM, ROM, cache) is fast; **secondary** is permanent and large	RAM, HDD, SSD
Output unit	Converts binary results into human-readable form	Monitor, printer, speaker

Unit	Size	Unit	Size	Memory hierarchy (fast & small → slow & large)
1 bit	0 or 1	1 MB	1024 KB	Registers → Cache → RAM → SSD/HDD
1 byte	8 bits	1 GB	1024 MB	Faster = costlier = smaller capacity
1 KB	1024 bytes	1 TB	1024 GB	RAM is **volatile**; ROM / disk are **non-volatile**

Program Counter (PC) and Important Registers

- **Program Counter (PC)** — holds the **address of the next instruction** to be executed. After every fetch it is automatically increased by 1 (instruction length).
- **Instruction Register (IR)** — holds the instruction currently being decoded.
- **Accumulator (ACC)** — stores intermediate results of the ALU.
- **MAR / MDR** — Memory Address Register / Memory Data Register: where to read/write, and what to read/write.

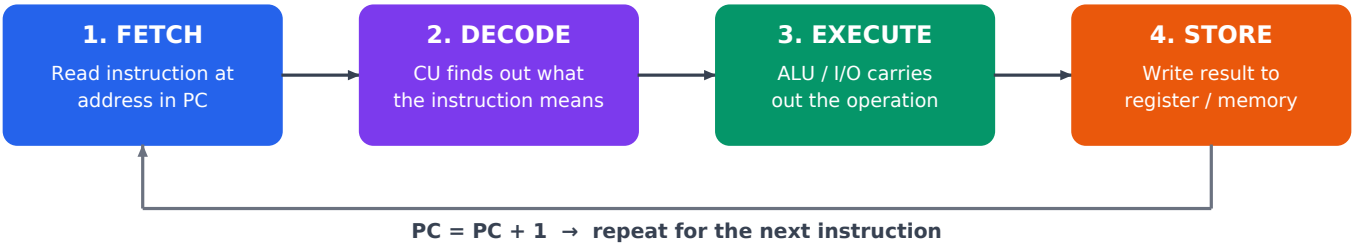


Fig 1.3 — Instruction cycle (Fetch–Decode–Execute)

1.3 Introduction to Programming Languages

A **programming language** is a set of words and rules used to give instructions to a computer. A **program** is a finite set of such instructions that solves a problem.

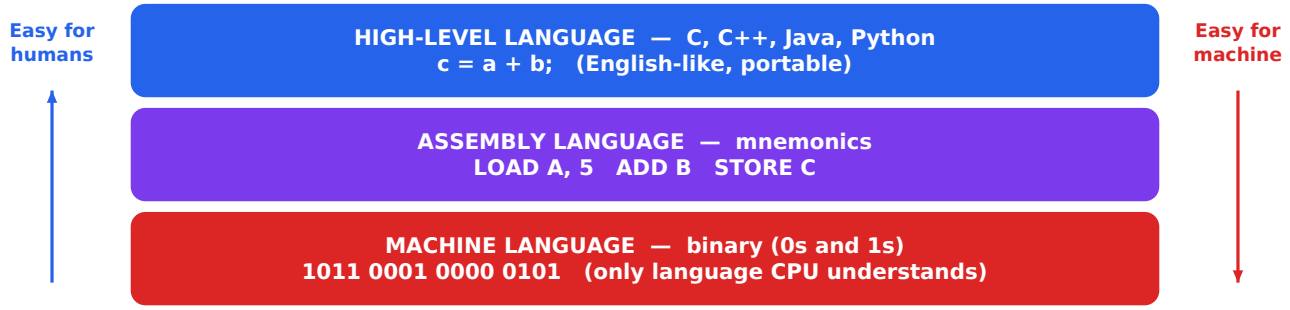


Fig 1.4 — Levels of programming languages (translators convert top → bottom)

	Machine	Assembly	High-level
Form	0s and 1s	Mnemonics (ADD, MOV)	English-like statements
Translator	None needed	Assembler	Compiler / Interpreter
Speed of run	Fastest	Very fast	Slightly slower
Portability	Machine dependent	Machine dependent	Machine independent
Ease	Very hard	Hard	Easy

Translator	How it works	Example
Compiler	Translates the **whole program** at once into machine code; reports all errors together	C, C++ (gcc)
Interpreter	Translates and runs **line by line**; stops at the first error	Python, JavaScript
Assembler	Converts assembly language to machine code	NASM, MASM

● Real-life Use
C is used to build operating systems (Linux, Windows kernel), embedded systems (washing machines, cars), compilers, databases and device drivers because it is **fast and close to hardware**.

1.4 Basics of a Computer Program

A C program is a collection of **functions**. Execution always starts from `main()`. Below is the smallest useful program with every part labelled.

sum.c

```c
#include <stdio.h>      /* 1. Preprocessor: add the I/O library    */

int main()              /* 2. main(): program starts here          */
{                       /* 3. Opening brace: block begins          */
    int a = 10, b = 20; /* 4. Declaration + initialization         */
    int sum;
    sum = a + b;        /* 5. Statement (ends with a semicolon)    */
    printf("Sum = %d\n", sum);  /* 6. Output                       */
    return 0;           /* 7. Tell OS: finished successfully       */
}                       /* 8. Closing brace: block ends            */
```

OUTPUT

```
Sum = 30
```

- **Comments** — `// single line` and `/* multi line */`; ignored by the compiler.
- **Keywords** are reserved words (`int, if, while, return`) — C has 32 of them.
- **Tokens** are the smallest units: keywords, identifiers, constants, strings, operators, punctuators.
- C is **case-sensitive**: `Sum` and `sum` are different. Every statement ends with **;**.

1.5 Algorithms, Flowcharts (Dia Tool) & Pseudocode

◆ Definition

An **algorithm** is a finite, step-by-step, unambiguous list of instructions to solve a problem. **Pseudocode** writes the algorithm in plain English with programming-like structure. A **flowchart** draws the algorithm using standard symbols.

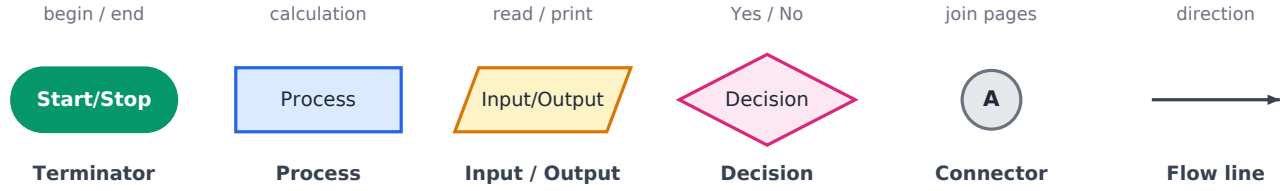
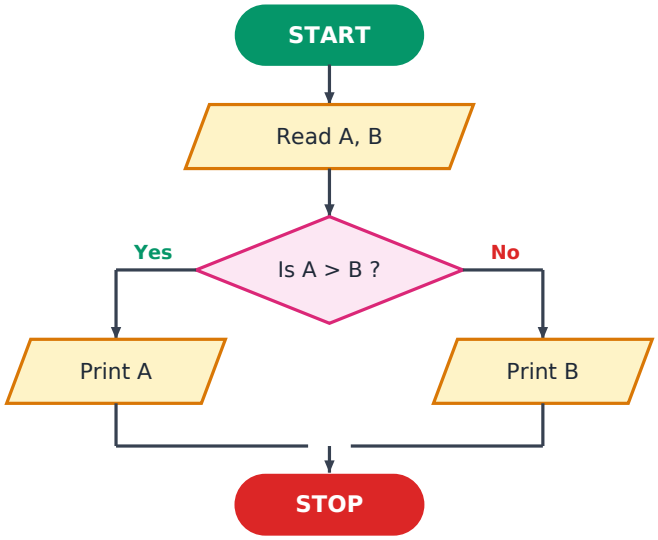


Fig 1.5 — Standard flowchart symbols



PSEUDOCODE

```
Algorithm: Largest of two numbers
1. START
2. READ A, B
3. IF A > B THEN
4.       PRINT A
5. ELSE
6.       PRINT B
7. END IF
8. STOP
```

Fig 1.6 — Flowchart + pseudocode: largest of two numbers

Drawing flowcharts with the Dia tool

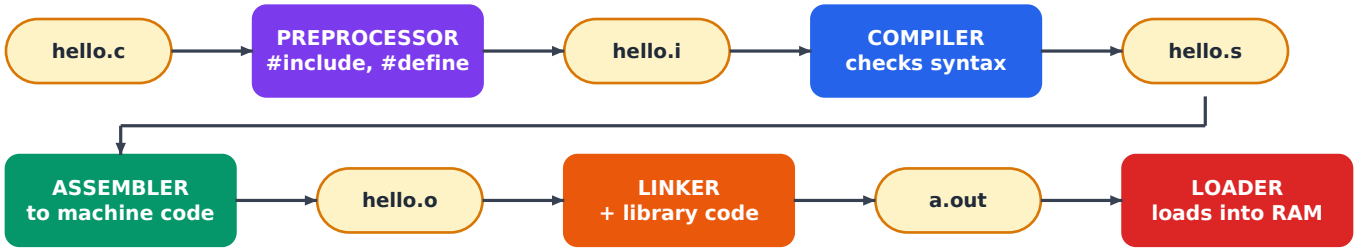
Dia is a free diagram editor used to draw flowcharts. Steps:

- **Open Dia** → in the toolbox drop-down choose the sheet **"Flowchart"**.
- **Drag symbols** (Terminal, Box, Diamond, Parallelogram) onto the canvas; **double-click** a symbol to type text.
- Select the **Line / Arrow** tool and join the symbols; connection points turn **red** when attached.
- Align with *Objects → Align*, then save as `.dia` and **File → Export** as PNG / PDF / SVG.

★ Exam Focus
Rules of flowcharting: one START and one STOP; flow runs top-to-bottom / left-to-right; a decision has **one input and two outputs** (Yes/No); every symbol has meaningful text; no crossing lines.

1.6 Introduction to Compilation and Execution

A computer cannot run C source code directly. It goes through **four stages** before it runs:



Source code → Object code → Executable → Running program
Fig 1.7 — Compilation and execution process in C (GCC)

Stage	Job
Preprocessing	Removes comments, expands `#include` and `#define`
Compilation	Checks syntax; converts to assembly. **Syntax errors** appear here
Assembling	Converts assembly into machine-level **object file** (.o / .obj)
Linking	Joins object file with library functions (e.g. printf) to make the **executable**

Terminal

```
gcc hello.c -o hello     # compile + link  (creates executable "hello")
./hello                  # run on Linux / macOS    (Windows: hello.exe)
gcc -Wall hello.c        # compile with all warnings switched on
```

Error type	When	Example
Syntax error	Compile time	Missing semicolon, misspelt keyword
Linker error	Link time	Calling a function that is not defined
Runtime error	While running	Divide by zero, invalid memory access
Logical error	Wrong output	Using + instead of * (program runs but result is wrong)

1.7 Primitive Data Types, Variables and Constants

A **data type** tells the compiler how much memory to reserve and what kind of value is stored.

Type	Size	Range (typical)	Format	Example
`char`	1 byte	−128 to 127	`%c`	`char g = 'A';`
`short`	2 bytes	−32,768 to 32,767	`%hd`	`short s = 100;`
`int`	4 bytes	-2.14×10^9 to 2.14×10^9	`%d`	`int age = 20;`
`long`	4 / 8 bytes	larger than int	`%ld`	`long pop = 9000000L;`
`unsigned int`	4 bytes	0 to 4,294,967,295	`%u`	`unsigned int n = 5;`
`float`	4 bytes	≈ 6–7 decimal digits	`%f`	`float pi = 3.14f;`
`double`	8 bytes	≈ 15–16 decimal digits	`%lf`	`double e = 2.71828;`
`void`	0	no value	–	`void f(void)`

Variables

A **variable** is a named memory location whose value can change. **Declare before use**:

Variables

```c
int age;              // declaration  (memory reserved, value = garbage)
age = 18;             // assignment
float marks = 87.5f;  // declaration + initialization
int x = 5, y = 10;    // multiple variables
```

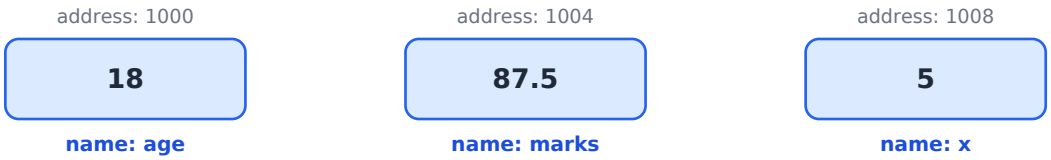


Fig 1.8 — A variable = name + address + value (box in memory)

- **Rules for identifier names:** letters, digits, underscore only; must **not start with a digit**; no spaces or special symbols; cannot be a keyword. Valid: `total_1, _x`. Invalid: `1st, my-var, int`.

Constants

Constants

```c
const float PI = 3.14159;   // const keyword: value cannot change
#define MAX 100             // symbolic constant (macro, no semicolon!)
// Literals:  10 (int)  3.5 (double)  'A' (char)  "Hi" (string)
```

1.8 Basic Input and Output

C uses `printf()` to print and `scanf()` to read. Both need `#include <stdio.h>`.

Specifier	Prints	Specifier	Prints
`%d / %i`	int	`%c`	single character
`%f`	float	`%s`	string
`%lf`	double	`%u`	unsigned int
`%.2f`	float with 2 decimals	`%x / %o`	hexadecimal / octal
`\n \t \\`	new line, tab, backslash	`%5d`	width 5, right aligned

Input & output

```c
#include <stdio.h>
int main() {
    int age; float h; char grade;
    printf("Enter age, height, grade: ");
    scanf("%d %f %c", &age, &h, &grade);
    printf("Age=%d  Height=%.1f  Grade=%c\n", age, h, grade);
    return 0;
}
```

OUTPUT

```
Enter age, height, grade: 19 5.8 A
Age=19  Height=5.8  Grade=A
```

> ⚠ **Common Mistake**
> Forgetting **&** in `scanf("%d", age)` — the program crashes. Use `%d` for int, `%f` for float. For single characters you can also use `getchar()` and `putchar()`; for strings with spaces use `fgets()`.

1.9 Operations (Operators) in C

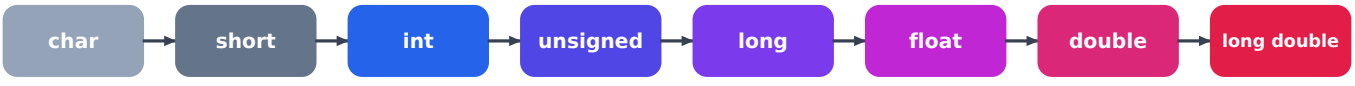
Category	Operators	Example (a=7, b=2)	Result
Arithmetic	+ − * / %	a / b , a % b	3 , 1 (integer division!)
Relational	< > <= >= == !=	a > b	1 (true)
Logical	&& || !	(a>5) && (b>5)	0 (false)
Assignment	= += −= *= /= %=	a += 3	a becomes 10
Increment / Decrement	++ --	a++ (post), ++a (pre)	a becomes 8
Conditional (ternary)	? :	max = (a>b) ? a : b;	max = 7
Bitwise	& | ^ ~ << >>	a & b , a << 1	2 , 14
sizeof	sizeof(type)	sizeof(int)	4

Priority	Operators (high → low)	Associativity
1	( ) ++ -- (postfix)	Left to right
2	! ~ ++ -- (prefix) sizeof, (type) cast	Right to left
3	* / %	Left to right
4	+ −	Left to right
5	< <= > >= then == !=	Left to right
6	&& then ||	Left to right
7	?: then = += −= ...	Right to left

> ✔ **Remember**
> **Post vs Pre:** if `a = 5`: `b = a++` gives **b=5, a=6** (use, then increase); `b = ++a` gives **b=6, a=6** (increase, then use). Example: `2 + 3 * 4 = 14` (multiplication first).

1.10 Type Conversion and Casting

When different data types are mixed in an expression, C converts them. There are two kinds:



smaller / lower rank — **automatic promotion →** — bigger / higher rank

Fig 1.9 — Implicit conversion: lower type is promoted to the higher type

	Implicit (automatic)	Explicit (type casting)
Done by	Compiler	Programmer
Syntax	None	`(type) expression`
Direction	Lower → higher (safe)	Any, may lose data

Conversion & casting

```c
int a = 5, b = 2;
float r1 = a / b;          // r1 = 2.0   (integer division first!)
float r2 = (float)a / b;   // r2 = 2.5   (a cast to float -> float division)
int   n  = 3.99;           // n = 3      (fraction truncated)
char  c  = 'A' + 1;        // c = 'B'    (ASCII 65 + 1 = 66)
```

OUTPUT

```
r1 = 2.0
r2 = 2.5
n  = 3
c  = B
```

1.11 Problem Solving Techniques

Algorithmic approach — the 6 steps

1 Understand the problem → **2 Plan** (inputs, outputs, process) → **3 Design** the algorithm / flowchart → **4 Code** in C → **5 Test & debug** → **6 Document & maintain**.

Characteristics of a good algorithm

Property	Meaning
Input	Zero or more well-defined inputs
Output	At least one output (the result)
Definiteness	Every step is clear and unambiguous
Finiteness	Must terminate after a finite number of steps
Effectiveness	Each step is simple enough to be carried out exactly (even by hand)
Correctness / Generality	Gives right answer for all valid inputs

Problem-solving strategies

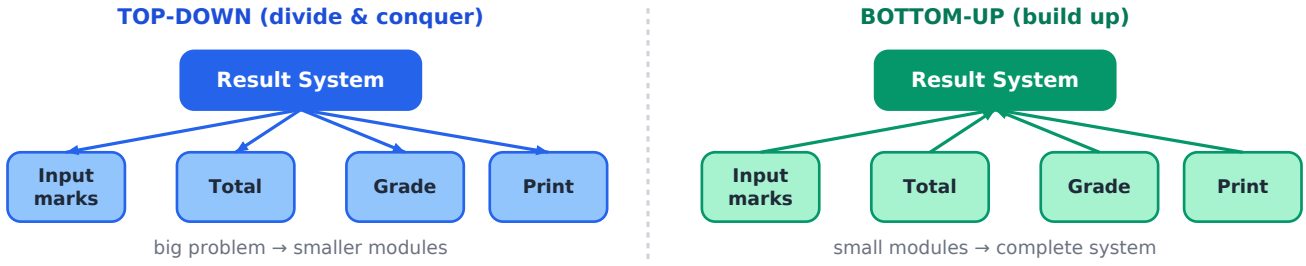


Fig 1.10 — Top-down vs bottom-up design

	Top-down approach	Bottom-up approach
Starts from	Whole problem, then breaks it into sub-problems	Small basic modules, then combines them
Used in	Procedural languages like C (functions)	Object-oriented design, reusing existing library code
Advantage	Easy to plan, understand and debug	Code reuse; modules can be tested early
Drawback	Needs full problem knowledge first	Overall design may be unclear at first

Time and space complexity of algorithms

Time complexity = how the running time grows with input size **n**. **Space complexity** = how the extra memory grows with **n**. We express both using **Big-O notation** (worst-case upper bound), ignoring constants.

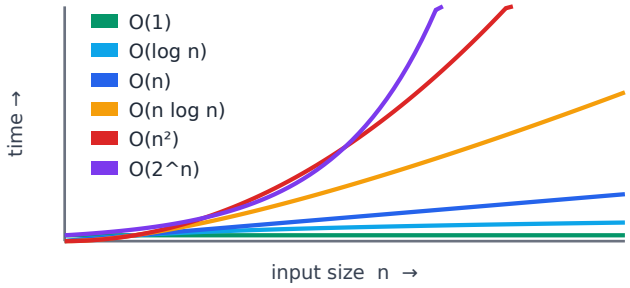


Fig 1.11 — Growth rate of common complexities

EXAMPLES

```
// O(1): one step
x = arr[0];

// O(n): loop runs n times
for (i = 0; i < n; i++)
    sum += i;

// O(n^2): nested loops
for (i = 0; i < n; i++)
  for (j = 0; j < n; j++)
    count++;
```

Big-O	Name	Example algorithm
O(1)	Constant	Access an array element
O(log n)	Logarithmic	Binary search
O(n)	Linear	Linear search, finding max
O(n log n)	Linearithmic	Merge sort
$O(n^2)$	Quadratic	Bubble sort, matrix addition
$O(2^n)$	Exponential	Recursive Fibonacci

★ Exam Focus

Space example: swapping two numbers using a temp variable needs 1 extra variable → **O(1)** space. Copying an array of n elements needs n extra cells → **O(n)** space. Rule: drop constants and lower-order terms — $3n^2 + 5n + 2$ becomes **$O(n^2)$**.

1.12 Solved Examples and Important Formulas

Problem	Formula	C expression
Area of circle	$A = \pi r^2$	`area = PI * r * r;`
Simple interest	SI = (P × R × T) / 100	`si = (p * r * t) / 100;`
Celsius → Fahrenheit	F = C × 9/5 + 32	`f = c * 9 / 5 + 32;`
Average of 3 numbers	Avg = (a + b + c) / 3	`avg = (a + b + c) / 3.0;`
Seconds → h : m : s	h = s/3600, m = (s%3600)/60, sec = s%60	`h = s / 3600;`

Sequential programs (no decisions, no loops)

```
#include <stdio.h>
#define PI 3.14159
int main() {
    float r = 5, area, P = 1000, R = 8, T = 2, si, c = 37, f;
    int a = 10, b = 20, temp, s = 3725;
    area = PI * r * r;            // area of circle
    si   = (P * R * T) / 100;     // simple interest
    f    = c * 9 / 5 + 32;        // temperature
    printf("Area=%.2f  SI=%.1f  F=%.1f\n", area, si, f);
    temp = a;  a = b;  b = temp;  // swap using a third variable
    printf("a=%d  b=%d\n", a, b);
    printf("%dh %dm %ds\n", s / 3600, (s % 3600) / 60, s % 60);
    return 0;
}
```

OUTPUT

```
Area=78.54  SI=160.0  F=98.6
a=20  b=10
1h 2m 5s
```

★ UNIT SUMMARY — QUICK REVISION

- A computer = **Input → CPU (ALU + CU + registers) → Output** with memory; the **PC** holds the next instruction's address.
- Languages: machine → assembly → high-level; translators are **assembler, compiler, interpreter**.
- C program: `#include`, `main()`, declarations, statements, `return 0;`
- Algorithm → **flowchart / pseudocode** → code → **preprocess → compile → assemble → link → run**.
- Data types: **char, int, float, double**; constants via `const` or `#define`; I/O via `printf / scanf` (use **&**).
- Remember: **int / int = int**; use a **cast** for decimals. Precedence: ( ) → * / % → + − → relational → logical → =
- Algorithm traits: input, output, **definiteness, finiteness, effectiveness**. Complexity: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$.

Q Exam-Oriented Questions

Short Answer (2 marks)

1. Define a computer. List its basic units.
2. What is the role of the Program Counter?
3. Differentiate between a compiler and an interpreter.
4. What is an algorithm? List any four characteristics.
5. Name any four flowchart symbols and their uses.
6. What is the difference between `const` and `#define`?
7. Why is `&` used in `scanf()`?
8. What is type casting? Give an example.
9. Distinguish top-down and bottom-up approaches.
10. What is the time complexity of linear search and binary search?

Long Answer / Programs (5–10 marks)

1. Explain the block diagram of a computer with a neat sketch.
2. Describe the generations of computers with their technologies.
3. Explain the phases of compilation and execution of a C program with a diagram.
4. Write an algorithm, pseudocode and flowchart to find the largest of three numbers.
5. Explain all C data types with size, range and format specifier.
6. Explain the types of operators in C with examples and the precedence table.
7. Explain implicit and explicit type conversion with examples.
8. Explain Big-O notation. Find the time complexity of nested loops and a single loop.